

LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT) II

Praktikum 4-5



Disusun Oleh :

Nama : Naila Rahma Chintya Aulya

NPM : 25782020

Kelas : 3A

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, A.Md

JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG

2026/2027

MODUL 4

Praktikum 4.5.1

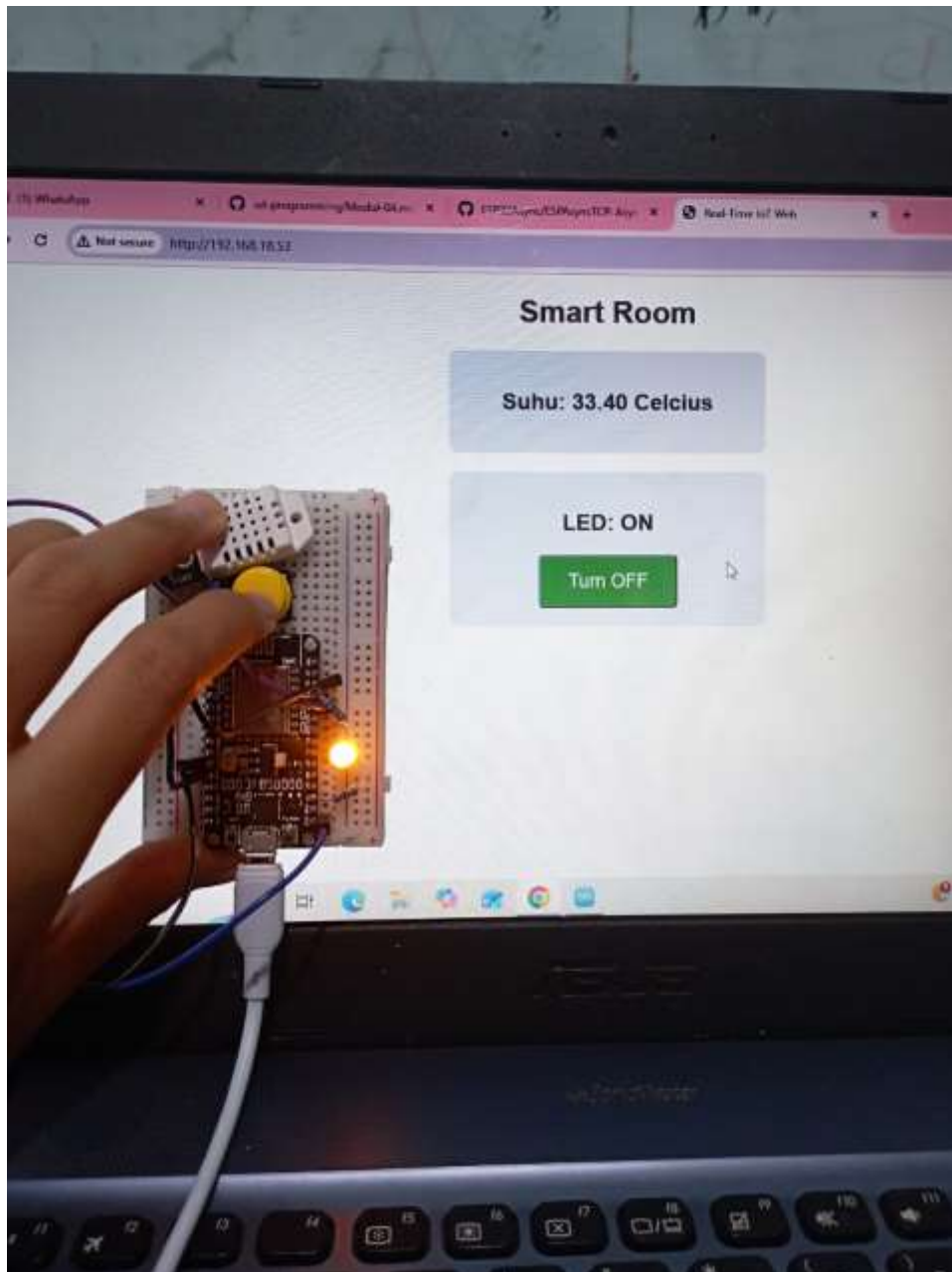
```
Blisk | Arduino IDE 2.3.10
File Edit Serial Tools Help
NodeMCU 1.0 (ESP-12...)
Blink.ino
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "Astra_tailor40";
7 const char* password = "Hailarahme";
8
9 // Konfiguriere Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(DHTPin, DHT22);
15
16 // Variabel 'Palacak Status' (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19
20 void setup() {
21   // Serial Monitor 'X'
22   Serial.begin(115200);
23   while (!Serial) continue;
24   Serial.println("NodeMCU 1.0 (ESP-12) Ready!");
25
26   // IP Address: 192.168.18.89
27   WiFi.begin(ssid, password);
28   while (WiFi.status() != WL_CONNECTED) {
29     delay(5000);
30     Serial.print("Connecting to WiFi... ");
31   }
32   Serial.println("Connected to WiFi!");
33   Serial.print("IP Address: ");
34   Serial.println(WiFi.localIP());
35
36   // WebServer
37   AsyncWebServer server(80);
38   server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {
39     request->send(200, "text/html", "NodeMCU 1.0 (ESP-12) Ready!");
40   });
41   server.on("/temp", HTTP_GET, [](AsyncWebServerRequest *request) {
42     float temp = dht->getTempC(2);
43     String tempStr = String(temp, 2);
44     request->send(200, "text/plain", tempStr);
45   });
46   server.on("/led", HTTP_GET, [](AsyncWebServerRequest *request) {
47     bool state = ledState;
48     String stateStr = state ? "ON" : "OFF";
49     request->send(200, "text/plain", stateStr);
50   });
51   server.begin();
52   Serial.println("WebServer started on port 80");
53 }
54
55 void loop() {
56   // Read DHT temperature
57   float tempC = dht->getTempC(2);
58   String tempStr = String(tempC, 2);
59   Serial.print("Temperature: ");
60   Serial.println(tempStr);
61
62   // Read Button State
63   bool buttonState = digitalRead(buttonPin) == LOW;
64   Serial.print("Button State: ");
65   Serial.println(buttonState ? "ON" : "OFF");
66
67   // Toggle LED State
68   ledState = !ledState;
69   digitalWrite(ledPin, ledState ? HIGH : LOW);
70   Serial.print("LED State: ");
71   Serial.println(ledState ? "ON" : "OFF");
72
73   // Delay
74   delay(1000);
75 }
```

Smart Room

Suhu: 33.40 Celcius

LED: OFF

Turn ON



15:39



192.168.18.53

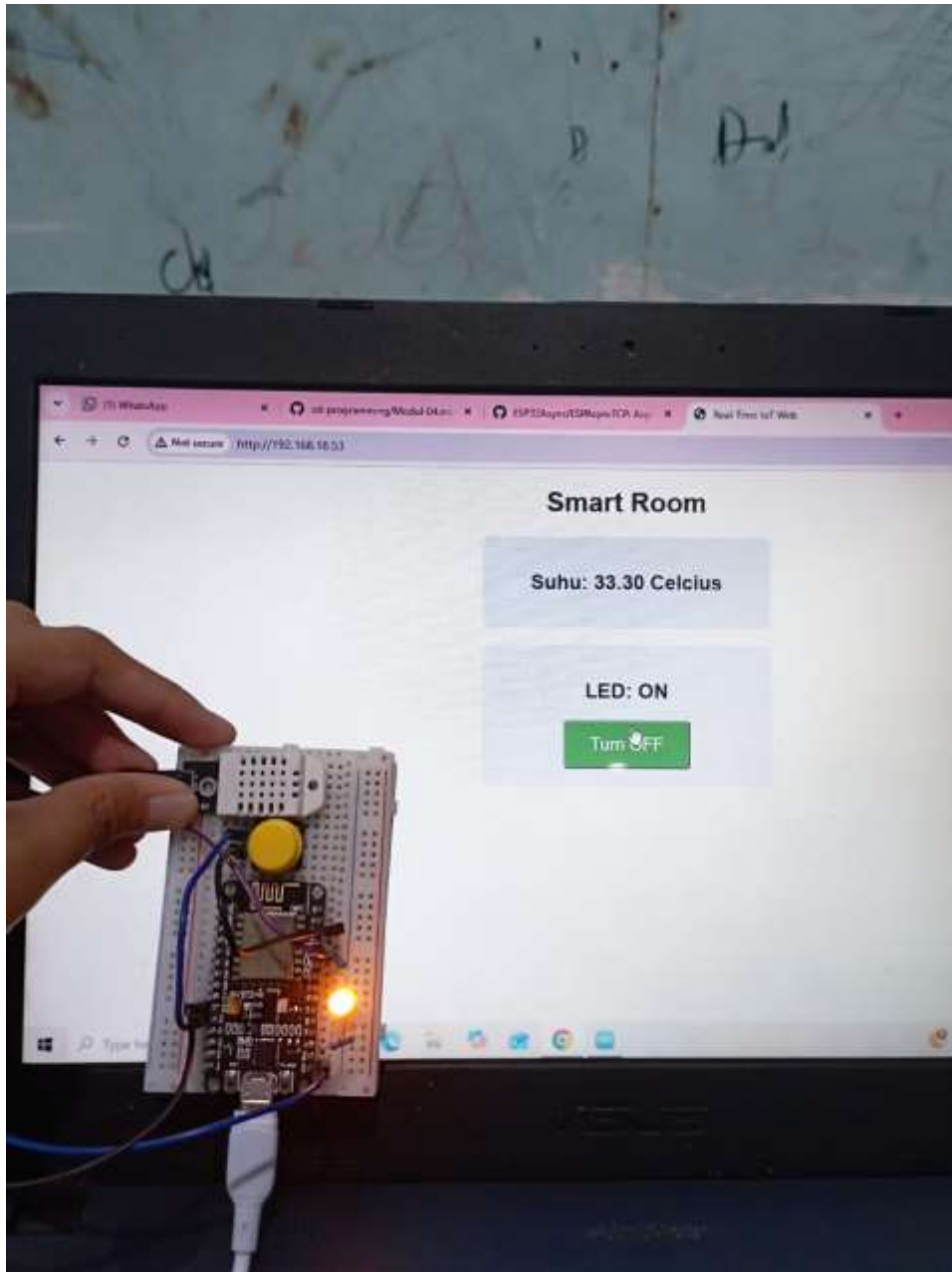


Smart Room

Suhu: 33.40 Celcius

LED: ON

Turn OFF



4.6.Latihan

Latihan 1 — Teori Jaringan

WebSockets lebih hemat bandwidth karena koneksi antara browser dan NodeMCU tetap terbuka. Jadi, ketika ada perubahan data, NodeMCU cukup mengirim data yang berubah melalui koneksi tersebut tanpa mengirim ulang seluruh halaman HTML. Berbeda dengan metode Meta Refresh pada HTTP yang harus melakukan request dan memuat ulang halaman secara berkala. WebSockets juga tidak membuat layar berkedip karena browser hanya memperbarui bagian data yang berubah menggunakan JavaScript, bukan memuat ulang seluruh halaman.

Latihan 2 — Analisis Waktu/Timer

Jika `delay(3000)` digunakan di dalam `void loop()`, NodeMCU akan berhenti melakukan proses selama 3 detik setiap kali `delay()` dijalankan. Selama waktu tersebut, respons terhadap tombol fisik dan komunikasi WebSocket dapat terlambat. Akibatnya, browser terasa lambat merespons, perubahan status LED tidak langsung terlihat, dan komunikasi real-time menjadi terganggu. Karena itu, program menggunakan `millis()` sebagai timer non-blocking sehingga NodeMCU tetap dapat menangani WebSocket, tombol, dan proses lainnya sambil menunggu waktu pembacaan sensor.

Latihan 3 — Modifikasi Front-End

Pada fungsi `onMessage(event)`, terdapat beberapa elemen HTML yang diperbarui oleh JavaScript. ID elemen yang menjadi target adalah:

`tempValue`

`ledStatus`

`toggleBtn`

Fungsinya:

- `tempValue` → menampilkan nilai suhu.
- `ledStatus` → menampilkan status LED, yaitu **ON** atau **OFF**.
- `toggleBtn` → mengubah tulisan tombol menjadi **Turn ON** atau **Turn OFF**, sekaligus mengubah class tombol.

Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

- Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--`";). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led": "...", "suhu": "...", "hum": "..."}).`
- Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

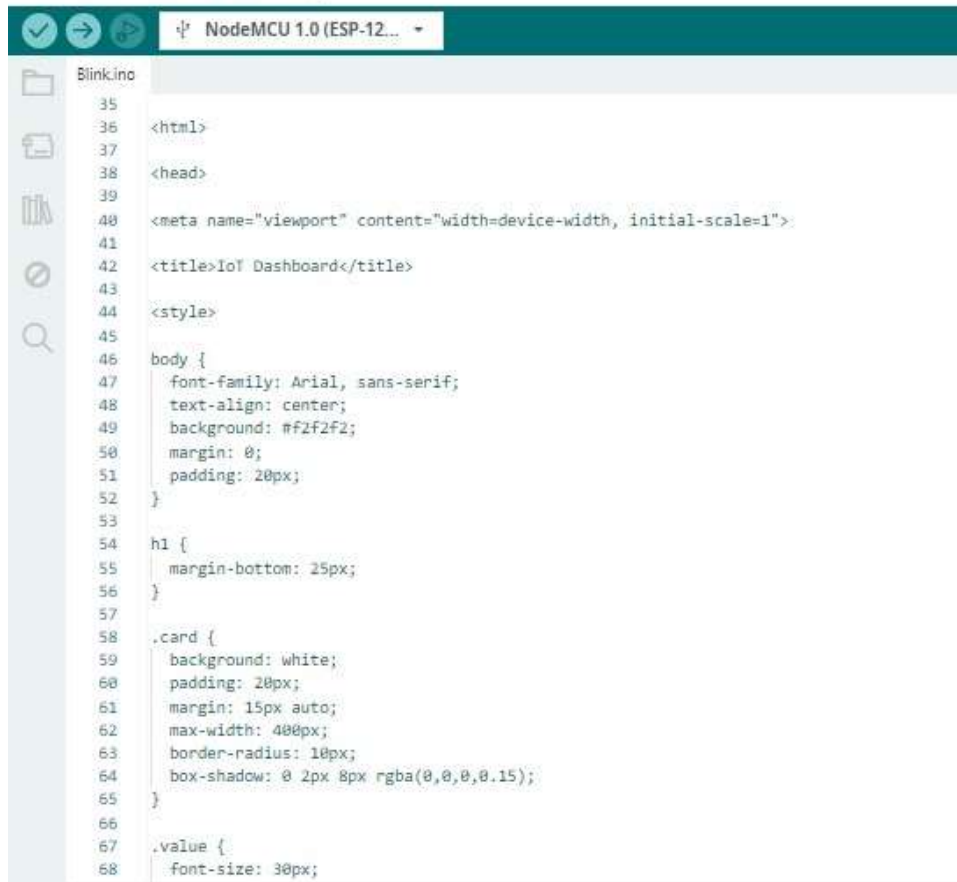
Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12... ▾

Blink.ino

```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "Astra_tailor-4G";
7 const char* password = "Nailarahma";
8
9 #define DHTPIN 2 // D4
10 #define DHTTYPE DHT22
11 const int buttonPin = 4; // D2
12 const int ledPin = 12; // D6
13
14 DHT dht(DHTPIN, DHTTYPE);
15
16 AsyncWebServer server(80);
17 AsyncWebSocket ws("/ws");
18
19 bool ledState = false;
20
21 String currentTemp = "--";
22 String currentHum = "--";
23
24 int buttonState = LOW;
25 int lastButtonState = LOW;
26
27 unsigned long lastDebounceTime = 0;
28 const unsigned long debounceDelay = 50;
29
30 unsigned long lastDHTRead = 0;
31 const unsigned long dhtInterval = 3000;
32
33 const char index_html[] PROGMEM = R"rawliteral(
34 <!DOCTYPE html>
```



```
35
36 <html>
37
38 <head>
39
40 <meta name="viewport" content="width=device-width, initial-scale=1">
41
42 <title>IoT Dashboard</title>
43
44 <style>
45
46   body {
47     font-family: Arial, sans-serif;
48     text-align: center;
49     background: #f2f2f2;
50     margin: 0;
51     padding: 20px;
52   }
53
54   h1 {
55     margin-bottom: 25px;
56   }
57
58   .card {
59     background: white;
60     padding: 20px;
61     margin: 15px auto;
62     max-width: 400px;
63     border-radius: 10px;
64     box-shadow: 0 2px 8px rgba(0,0,0,0.15);
65   }
66
67   .value {
68     font-size: 30px;
```



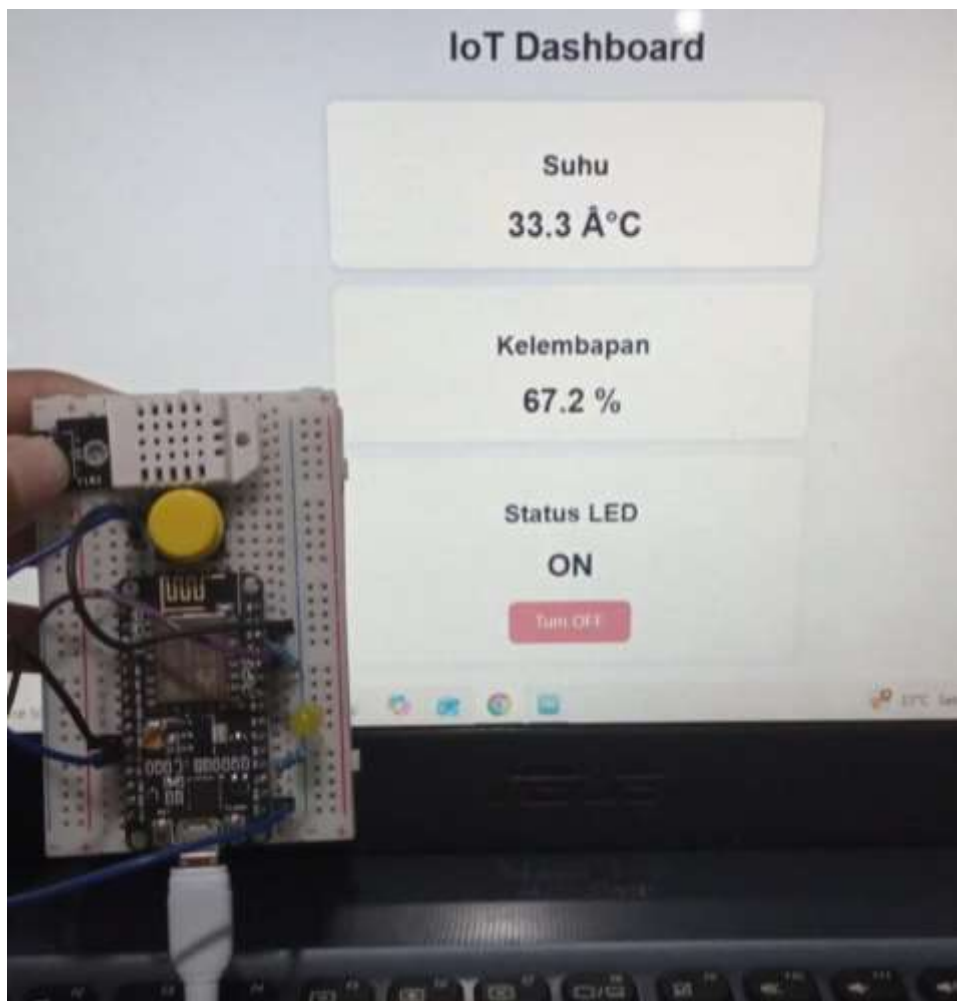
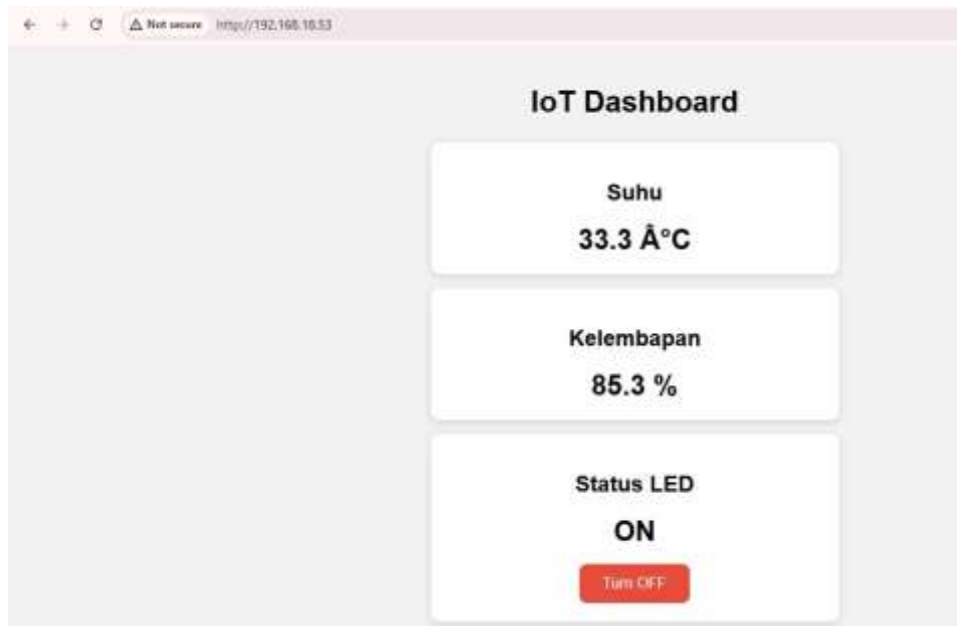
```
69     font-weight: bold;
70 }
71
72 button {
73     padding: 12px 25px;
74     border: none;
75     border-radius: 8px;
76     font-size: 16px;
77 }
78
79 .btn-on {
80     background: #e74c3c;
81     color: white;
82 }
83
84 .btn-off {
85     background: #2ecc71;
86     color: white;
87 }
88
89 </style>
90
91 </head>
92
93
94 <body>
95
96 <h1>IoT Dashboard</h1>
97
98
99 <!-- SUHU -->
100
101 <div class="card">
```

```
Blinking
103 <h2>Suhu</h2>
104
105 <div class="value">
106 <span id="tempValue">--</span> °C
107 </div>
108
109 </div>
110
111
112 <!-- KELEMBAPAN -->
113
114 <div class="card">
115
116 <h2>Kelembapan</h2>
117
118 <div class="value">
119 <span id="humValue">--</span> %
120 </div>
121
122 </div>
123
124
125 <!-- LED -->
126
127 <div class="card">
128
129 <h2>Status LED</h2>
130
131 <div class="value">
132
133 <span id="ledStatus">OFF</span>
134
135 </div>
136
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

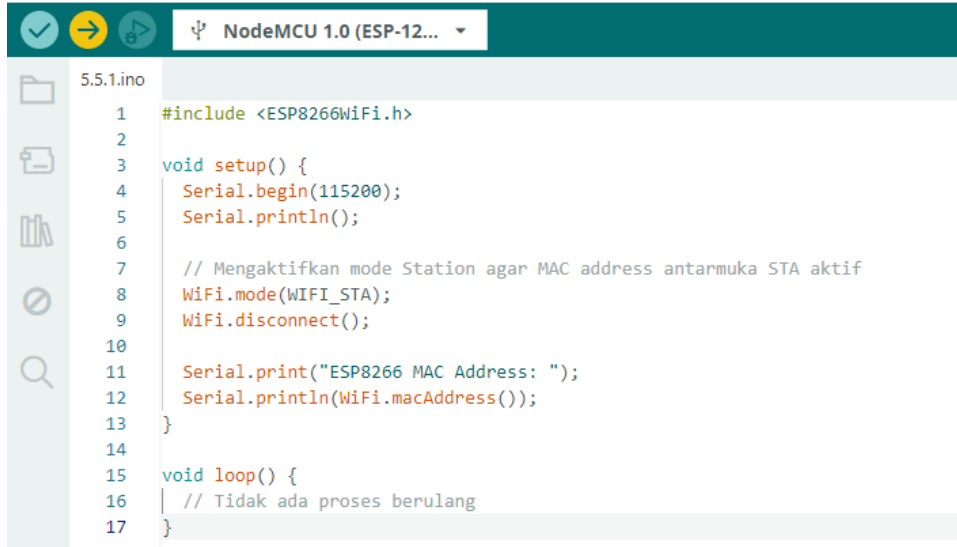
```
Data dikirim: {"led":"0","suhu":"32.9","hum":"80.5"}
Suhu: 33.0 °C | Kelembapan: 79.6 %
Data dikirim: {"led":"0","suhu":"33.0","hum":"79.6"}
Suhu: 33.0 °C | Kelembapan: 80.1 %
Data dikirim: {"led":"0","suhu":"33.0","hum":"80.1"}
Suhu: 33.1 °C | Kelembapan: 81.0 %
Data dikirim: {"led":"0","suhu":"33.1","hum":"81.0"}
Data dikirim: {"led":"1","suhu":"33.1","hum":"81.0"}
```



5.5 Praktikum

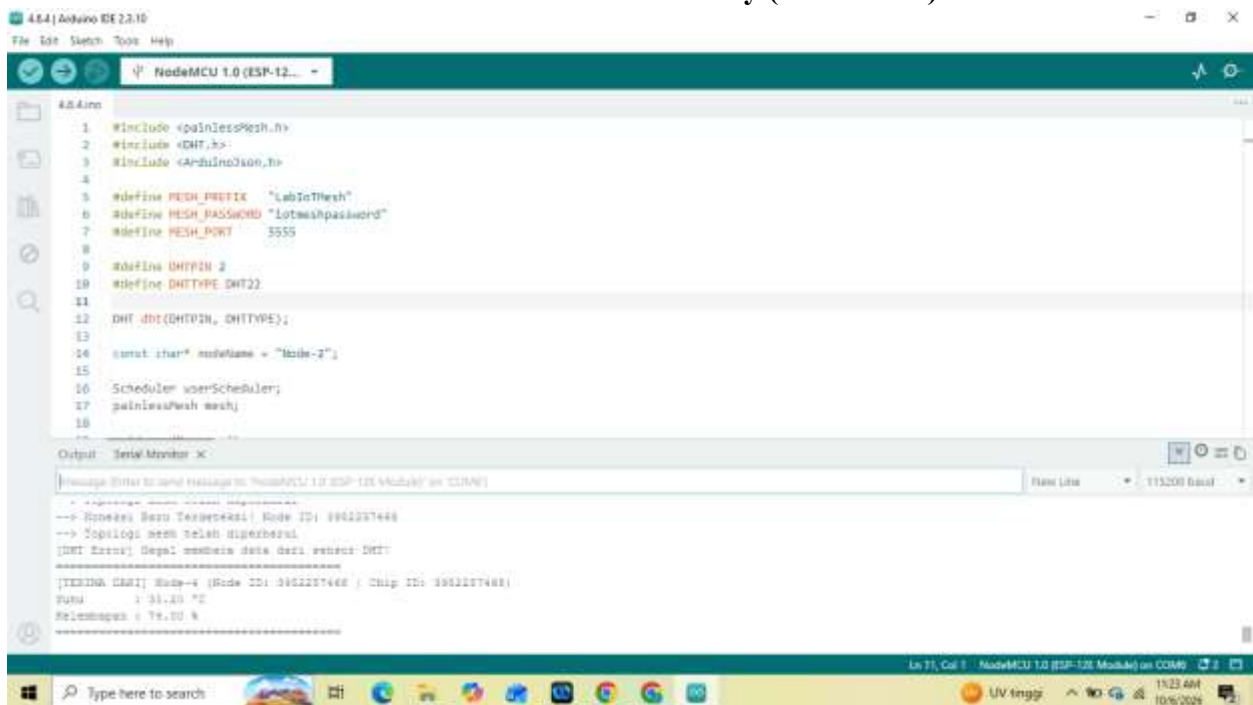
5.5.1 | Arduino IDE 2.3.10

File Edit Sketch Tools Help



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh"
6 #define MESH_PASSWORD "lotsamshpassword"
7 #define MESH_PORT 5555
8
9 #define DHTPIN 2
10 #define DHTTYPE DHT22
11
12 DHT dht(DHTPIN, DHTTYPE);
13
14 const char* nodeName = "Node-2";
15
16 #include <Scheduler.h>
17 #include <Scheduler.h>
18 #include <Scheduler.h>
```

Output Serial Monitor X:

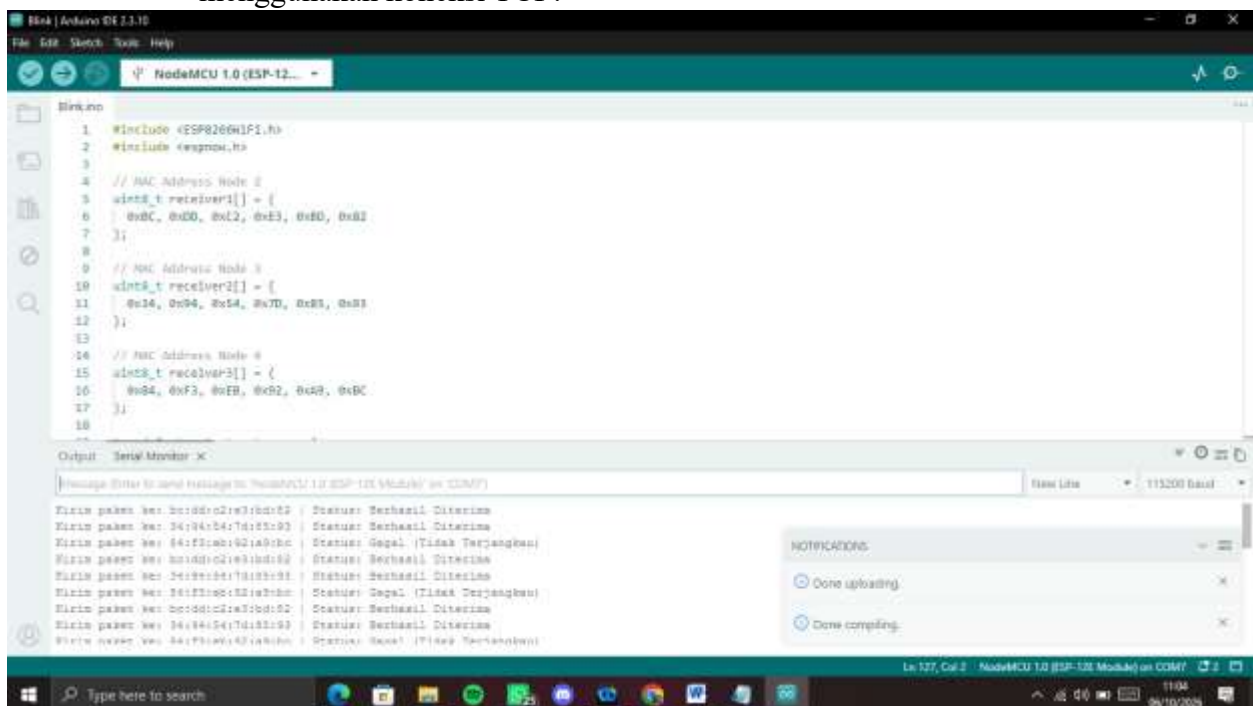
```
Message (0x10) to send Message to 'NodeMCU 1.0 (ESP-12E Module)' on '127.0.0.1'
--> Received Baru Tersepetasi! Node ID: 8862287468
--> Topologi mesh telah diproses!
[DHT Error] Segal metode data dari sensor DHT:
=====
[TERIMA DATA] Node-4 (Node ID: 8862287468 / Chip ID: 8862287468)
Puhu : 31.20 °C
Kelembapan : 78.00 %
```

5.5.3 Praktikum: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)



Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!



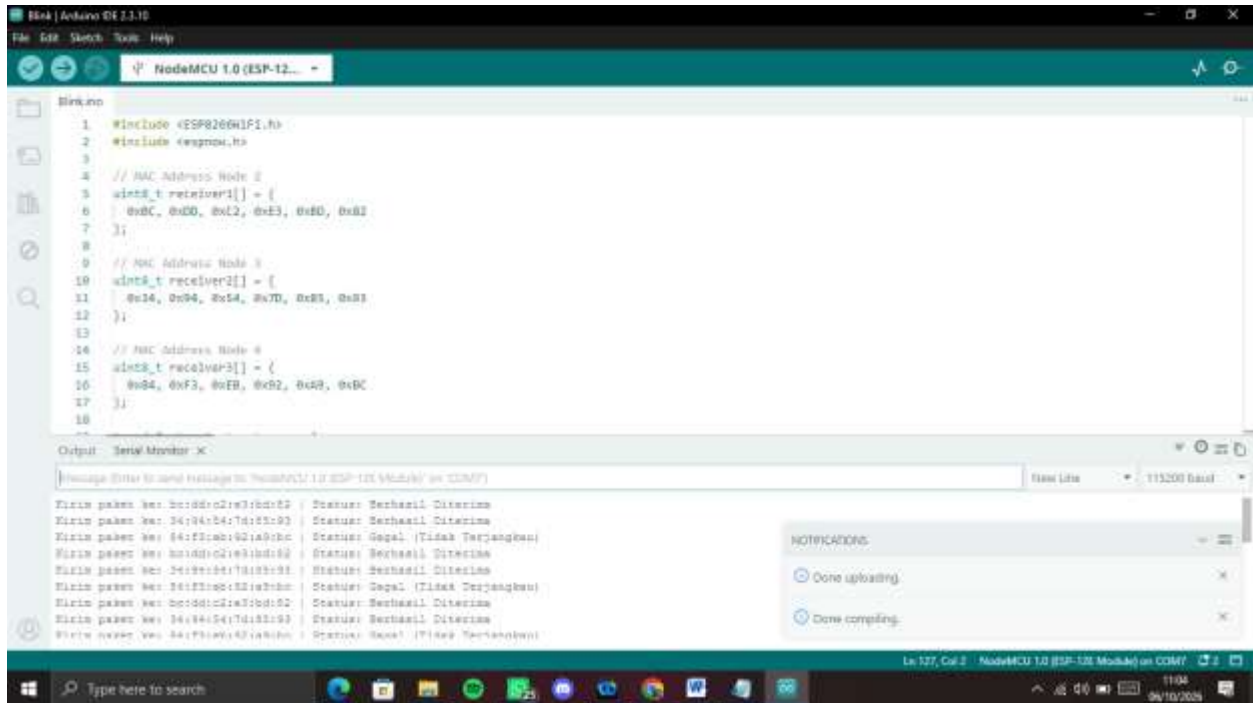
The screenshot shows the Arduino IDE interface. The code in the main editor includes the following MAC addresses:

```
1 // MAC Address Node 2
2 #include <ESP8266WiFi.h>
3 #include <espnow.h>
4
5 // MAC Address Node 2
6 uint8_t receiver1[] = {
7   0x0C, 0x00, 0x12, 0xE3, 0x00, 0x03
8 };
9
10 // MAC Address Node 3
11 uint8_t receiver2[] = {
12   0x34, 0x04, 0x54, 0x7D, 0x03, 0x03
13 };
14
15 // MAC Address Node 4
16 uint8_t receiver3[] = {
17   0x84, 0xF3, 0xEB, 0xB2, 0x4B, 0x0C
18 };
19
```

The Serial Monitor output shows the following status messages:

From packet to:	Status
0x0C:0x00:0x12:0xE3:0x00:0x03	Status: Berhasil Diterima
0x34:0x04:0x54:0x7D:0x03:0x03	Status: Berhasil Diterima
0x84:0xF3:0xEB:0xB2:0x4B:0x0C	Status: Gagal (Tidak Terjangkau)
0x0C:0x00:0x12:0xE3:0x00:0x03	Status: Berhasil Diterima
0x34:0x04:0x54:0x7D:0x03:0x03	Status: Berhasil Diterima
0x84:0xF3:0xEB:0xB2:0x4B:0x0C	Status: Gagal (Tidak Terjangkau)
0x0C:0x00:0x12:0xE3:0x00:0x03	Status: Berhasil Diterima
0x34:0x04:0x54:0x7D:0x03:0x03	Status: Berhasil Diterima
0x84:0xF3:0xEB:0xB2:0x4B:0x0C	Status: Gagal (Tidak Terjangkau)

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:
 - i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
 - ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!



- **Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
 - ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetry dari Node <chipId> aman.
 - iii. Tuliskan blok fungsi receivedCallback(uint32_t from, String &msg) hasil modifikasi Anda!

